📘 Programação para Internet II

📅 Aula 01 — Revisão HTML, CSS e Introdução ao JavaScript

🎯 Objetivo da Aula

- Revisar HTML e CSS
- Entender o papel do JavaScript
- Criar um pequeno projeto base
- Testar o nível da turma

🌐 O Trio da Web

- 🧱 **HTML** → Estrutura
- 🎨 **CSS** → Aparência
- ⚙️ **JavaScript** → Comportamento

📚 Documentações Oficiais (Guarde esses links)

- 📘 HTML (MDN): https://developer.mozilla.org/pt-BR/docs/Web/HTML
- 🎨 CSS (MDN): https://developer.mozilla.org/pt-BR/docs/Web/CSS
- ⚙️ JavaScript (MDN): https://developer.mozilla.org/pt-BR/docs/Web/JavaScript
- 🌐 JavaScript Info (Didático): https://javascript.info/
- 🧠 W3Schools: https://www.w3schools.com/

🧠 O que é JavaScript? (Introdução Teórica)

> **JavaScript é uma linguagem de programação que roda principalmente no navegador e serve para tornar as páginas interativas e dinâmicas.**

Com JavaScript, é possível:

- Reagir a cliques de botões
- Alterar textos e cores da página
- Validar formulários
- Criar animações
- Buscar dados de outros sites (APIs)
- Criar jogos e sistemas completos no navegador

⚙️ Como o JavaScript funciona no navegador?

1. O navegador carrega o HTML
2. Aplica o CSS
3. Executa o JavaScript

O JavaScript:

- Pode **ler** o HTML
- Pode **modificar** o HTML
- Pode **modificar** o CSS
- Pode **reagir às ações do usuário**

🧠 Onde o JavaScript roda?

- Principalmente no **navegador**
- Também pode rodar no **servidor** usando **Node.js**

Hoje o JavaScript é usado:

- Front-end (sites)
- Back-end (servidores)
- Apps
- Jogos
- Sistemas completos

🧠 Como o JavaScript é executado?

O navegador possui um **motor JavaScript** (ex: V8 no Chrome) que:

- Lê o código
- Interpreta
- Executa linha por linha

🛠️ Check-up de Ferramentas

- VS Code instalado e atualizado.
- Navegador moderno (Chrome, Edge ou Firefox).
- Extensão:
    - Live Server: Permite visualizar as mudanças no navegador em tempo real assim que você salva o arquivo.
    - Prettier: Formata o seu código automaticamente (HTML, CSS e JS), mantendo-o organizado e padronizado.

- CSS Peek: Ajuda na identificação rápida de IDs e Classes CSS diretamente pelo arquivo HTML, facilitando a navegação entre os arquivos.

🧱 Parte 1 — Criando o Projeto Base

Crie uma pasta chamada:

aula01-projeto

Dentro dela, crie os arquivos:

- index.html
- style.css
- script.js

📄 1️⃣ Criando o HTML (index.html)

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Minha Primeira Página</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="card">
    <h1>Programação para Internet II</h1>
    <p id="texto">Aula 01 - Revisão HTML, CSS e JS</p>
    <button onclick="mostrarMensagem()">Clique aqui</button>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

🧠 Revisão: Principais Tags HTML

- `<h1>` → título
- `<p>` → parágrafo
- `<button>` → botão
- `<div>` → divisão/bloco
- `<link>` → conecta o CSS
- `<script>` → conecta o JavaScript

🧱 HTML Semântico (HTML5)

📚 Referência: https://www.w3schools.com/html/html5_semantic_elements.asp

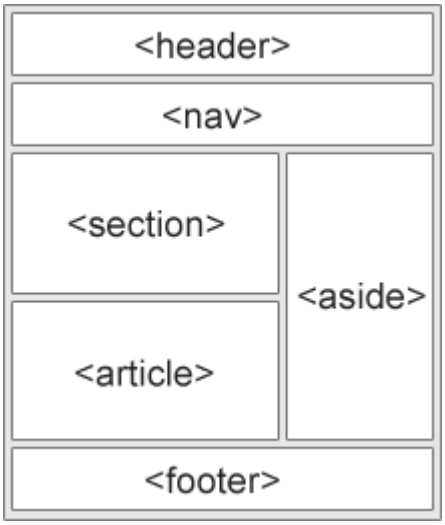
🧠 O que é HTML Semântico?

> HTML semântico é o uso de **tags que possuem significado**, ou seja, que descrevem claramente **o papel do conteúdo** dentro da página.

Em vez de usar apenas `<div>`, usamos tags que dizem **o que aquele bloco é**.

🧩 Principais Tags Semânticas

- `<header>` → Cabeçalho da página ou de uma seção
- `<nav>` → Menu de navegação
- `<main>` → Conteúdo principal
- `<section>` → Seção de conteúdo
- `<article>` → Conteúdo independente (post, notícia, card)
- `<aside>` → Conteúdo lateral
- `<footer>` → Rodapé



🖼️ Estrutura Semântica de uma Página

```
<body>
  <header>Topo</header>
  <nav>Menu</nav>

  <main>
    <section>
      <article>Conteúdo</article>
    </section>
```

```
    <aside>Barra lateral</aside>
  </main>

  <footer>Rodapé</footer>
</body>
```

🔁 Comparação: Div vs Semântico

❌ Antigo:

```
<div id="topo"></div>
<div id="menu"></div>
<div id="conteudo"></div>
<div id="rodape"></div>
```

☑ Moderno:

```
<header></header>
<nav></nav>
<main></main>
<footer></footer>
```

☑ Por que usar HTML Semântico?

- Melhor para **Google e SEO**
- Melhor para **leitores de tela (acessibilidade)**
- Código mais **organizado e profissional**
- Mais fácil de manter

🧪 Mini-desafio

> Troque a `<div class="card">` do nosso projeto por:

```
<main class="card">
```

E observe: **continua funcionando igual**, mas agora com significado semântico.

🎨 2 Criando o CSS (style.css)

🟢 Estilo 1 — Layout Básico

```
body {
  font-family: Arial;
  background-color: #eeeeee;
  text-align: center;
  padding-top: 50px;
}

.card {
  background-color: white;
  width: 400px;
  margin: auto;
  padding: 20px;
  border-radius: 10px;
}
```

🟢 Estilo 2 — Melhorando Visual

```
h1 {
  color: #0055aa;
}

button {
  padding: 10px 20px;
  border: none;
  background-color: #0055aa;
  color: white;
  border-radius: 5px;
  cursor: pointer;
}

button:hover {
  background-color: #003366;
}
```

🟢 Estilo 3 — Efeito Visual

```
.card {
  box-shadow: 0 0 10px rgba(0,0,0,0.2);
}
```

🧠 Revisão: Conceitos de CSS

- Seletor por tag: `h1 { }`
- Seletor por classe: `.card { }`
- Seletor por id: `#texto { }`
- Propriedades:
  - `color`
  - `background-color`
  - `margin`
  - `padding`
  - `border-radius`
  - `box-shadow`

⚙️ 3️⃣ Criando o JavaScript (script.js)

```
function mostrarMensagem() {
  alert("JavaScript está funcionando!");
}
```

🧪 Testando

1. Abra o `index.html`
2. Clique no botão
3. Veja o alerta aparecer

🧠 O que aprendemos até agora?

- HTML cria a estrutura
- CSS deixa bonito
- JS cria interatividade

🧩 Desafio 1 (Em Sala)

- Trocar as cores
- Mudar a fonte
- Alterar o tamanho do card
- Centralizar melhor o conteúdo

🧩 Desafio 2 (Para quem terminar rápido)

```
function mostrarMensagem() {
  document.getElementById("texto").innerText = "O texto foi alterado com
JavaScript!";
}
```

🧠 Explicação Rápida

- `document.getElementById()` → seleciona um elemento
- `.innerText` → muda o texto

🏁 DESAFIO FINAL — Aula 01

```
📐 Especificação Visual do Layout (Legenda de Reprodução)
O objetivo é reproduzir o layout o mais fiel possível usando HTML e CSS.
http://localhost:5500/exemplo_professor.html
```

🎨 🎯 Paleta de Cores

Elemento Cor

- Fundo da página #eeeeee
- Card #ffffff
- Título #0055aa
- Texto #333333
- Botão principal #0055aa
- Texto botão principal #ffffff
- Botão secundário #cccccc
- Texto botão secundário #333333

📏 📐 Medidas e Espaçamentos

Item Valor aproximado

- Largura do card 360px
- Espaçamento interno 20px
- Espaçamento superior pág 60px
- Altura da imagem 200px
- Espaço entre botões 10px
- Borda arredondada card 12px

- Borda arredondada botão 6px

🖼️ Sombra do Card

- box-shadow: 0 0 15px rgba(0,0,0,0.2);

🔤 Fonte

- Fonte: Arial
- Título: maior que o texto normal
- Texto: tamanho padrão

🧱 Estrutura Visual Esperada

- Ordem dos elementos dentro do card: Imagem (ocupando 100% da largura) Área de conteúdo com:
- Título
- Texto
- Área dos botões
- Dois botões lado a lado

🖼️ Imagem

- Pode usar: https://picsum.photos/600/300
- Deve: Ocupar 100% da largura do card
- Ter aproximadamente 200px de altura

🧩 Comportamento Visual Esperado

- Card centralizado horizontalmente
- Conteúdo centralizado dentro do card
- Nada colado nas bordas
- Layout limpo e organizado

📋 Regras Técnicas

- ☑️ Usar apenas HTML e CSS
- ☑️ Separar em dois arquivos: index.html e style.css
- ❌ Não usar Bootstrap, Tailwind, etc
- ❌ Não usar JavaScript

🧠 O que será avaliado

- Fidelidade visual ao modelo
- Organização do código
- Uso correto de classe
- Uso de espaçamentos
- Centralização correta
- Capricho visual

🏆 Critério de Excelência

- Não precisa ser idêntico, mas deve estar claramente baseado no modelo.

Avaliação Diagnóstica

Observe:

- Quem sabe mexer em HTML
- Quem entende CSS
- Quem nunca programou

Próxima Aula

> Entender o Ecossistema Git/GitHub; Vamos começar JavaScript do zero; Variáveis e Tipos de Dados.